Time Series Project: IPI Food Industries

```
knitr::opts_chunk$set(
  echo = TRUE,
  warning = TRUE,
  message = TRUE,
  fig.align = "center",
  fig.width = 10,
  fig.height = 6
)

required_pkgs <- c(
  "readr", "dplyr", "ggplot2", "lubridate",
  "forecast", "tseries", "ellipse", "knitr", "ellipse"
)

missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
if (length(missing_pkgs) > 0) {
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}

library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(tseries)
library(forecast)
library(knitr)
library(ellipse)
```

```
##
## Attaching package: 'ellipse'

## The following object is masked from 'package:graphics':
##
##     pairs
```

1 Part I: The data

1.1 1. What does the chosen series represent?

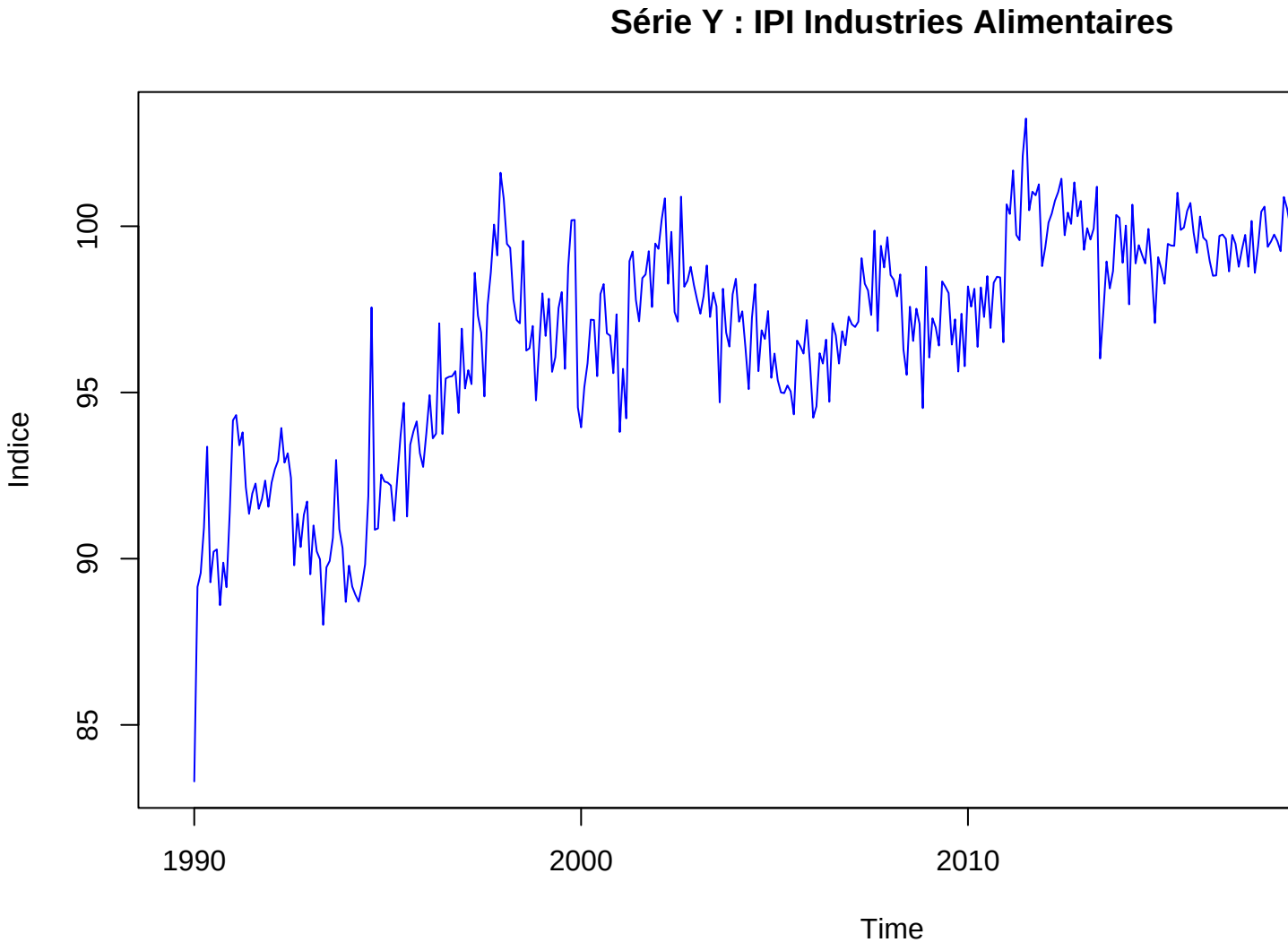
The dataset is an Industrial Production Index (IPI) for the food industries sector in France, with monthly frequency, seasonally and working-day adjusted (CVS-CJO), based at 100 in 2021. The variable is an index and is unit-free. Since the series is already CVS-CJO, we do not apply additional seasonal adjustment.

Modelling context.

- As an index (based at 100 in 2021), values above (below) 100 indicate production above (below) its 2021 average level for the sector.
- The CVS-CJO correction removes most deterministic seasonal patterns and calendar effects; consequently, we do not expect pronounced remaining seasonality and we focus on non-seasonal ARIMA/ARMA dynamics.
- Because the index is strictly positive over the sample, the analysis can be carried out on $Z_t = \log(Y_t)$; first differences of Z_t have a growth-rate interpretation and often provide variance stabilization.

1.2 2 and 3. Transform the series to make it stationary if necessary whith graphical representation before and after transformation (placed here for convenience of the study)

```r
df <- read.csv("data/valeurs_mensuelles.csv", sep = ";", skip = 4, header = FALSE, stringsAsFactors = F

df <- df[, 1:2]
colnames(df) <- c("date_brute", "valeur")

df <- df[df$valeur != "" & !is.na(df$valeur), ]

# On ajoute "-01" pour créer un format AAAA-MM-JJ lisible par R
df$date <- as.Date(paste0(df$date_brute, "-01"))
df <- df[order(df$date), ]

# On récupère l'année et le mois de début
start_y <- as.numeric(format(df$date[1], "%Y"))
start_m <- as.numeric(format(df$date[1], "%m"))

Y <- ts(df$valeur, start = c(start_y, start_m), frequency = 12)

plot(Y, main = "Série Y : IPI Industries Alimentaires", ylab = "Indice", col = "blue")
```

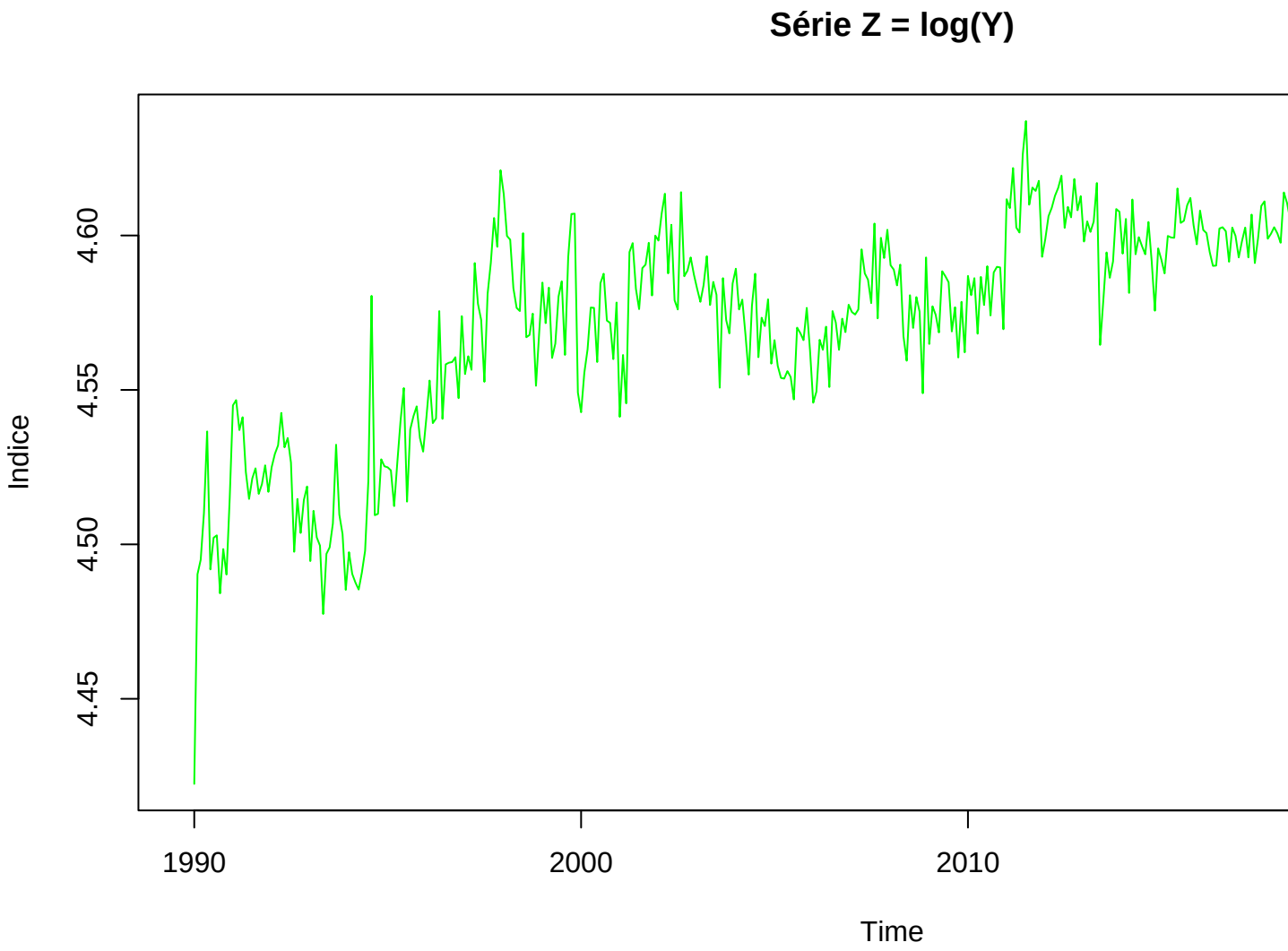


Visual inspection of the raw series Y_t shows that the amplitude of fluctuations increases proportionally with the level of the index. To address this heteroscedasticity, we apply a natural logarithm transformation:

$$Z_t = \ln(Y_t) \tag{1}$$

Justification: This transformation stabilizes the variance and converts multiplicative growth patterns into additive ones. Economically, the first difference of the logged series approximates the monthly growth rate, a standard metric for industrial production analysis.

```
Z <- log(Y)
plot(Z, main = "Série Z = log(Y)", ylab = "Indice", col = "green")
```

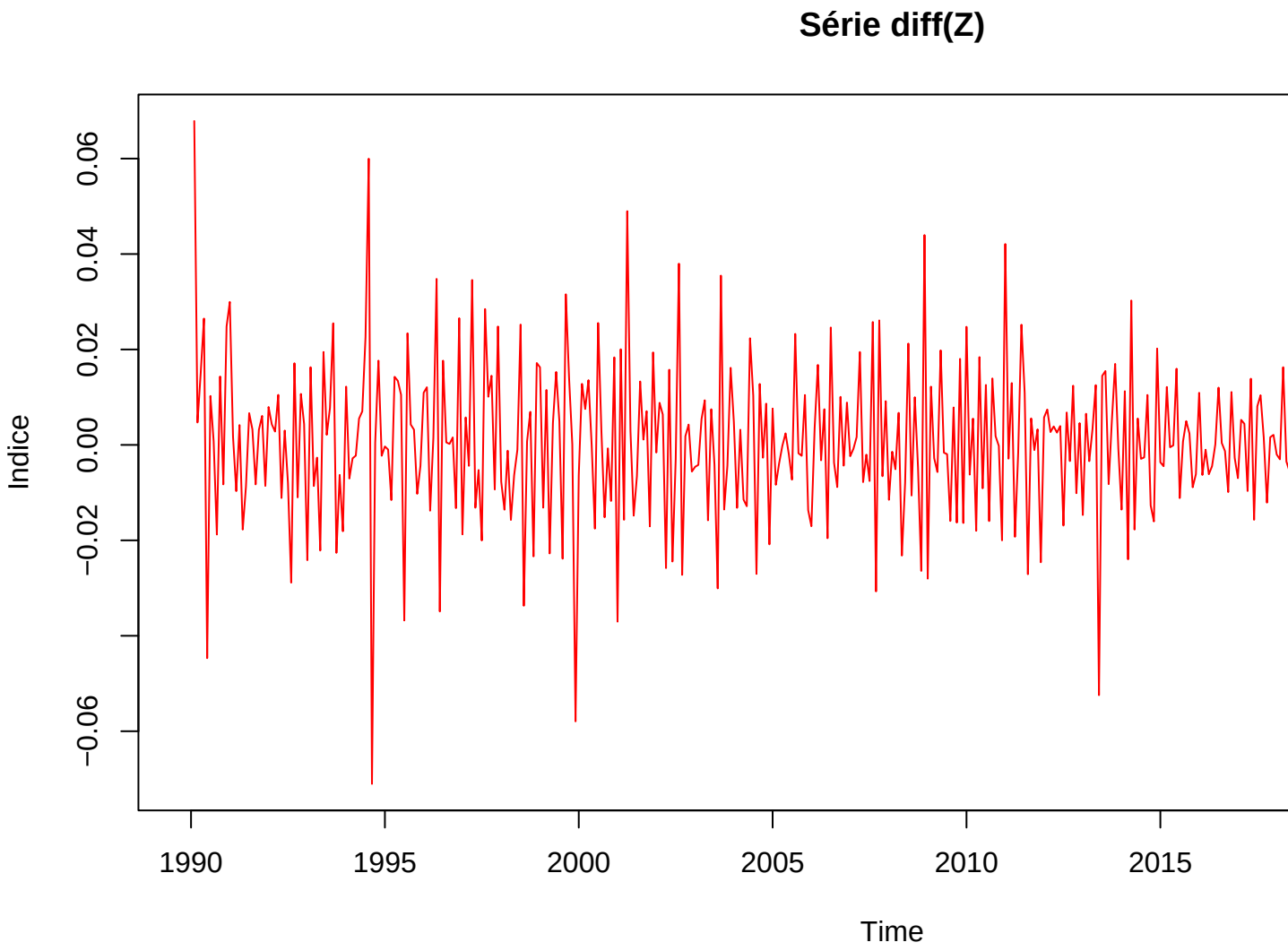


The series Z_t exhibits a clear upward long-term trend, indicating that the mean is not constant over time. To remove this stochastic trend and eliminate the unit root, we apply first-order differencing ($d = 1$):

$$X_t = \Delta Z_t = \ln(Y_t) - \ln(Y_{t-1}) = \nabla^1 \log(Y_t) \tag{2}$$

Justification: This process transforms the series into a "Difference-Stationary" process. The resulting series X_t represents the monthly innovations in production, which is a requirement for identifying ARMA (p, q) processes.

```
X <- diff(Z, differences = 1)
plot(X, main = "Série diff(Z)", ylab = "Indice", col = "red")
```



To rigorously confirm that the transformed series X_t is stationary, we performed the following tests based on the processed data:

```
adf.test(X)
```

```
## Warning in adf.test(X): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  X
## Dickey-Fuller = -11.349, Lag order = 7, p-value = 0.01
## alternative hypothesis: stationary
```

```r
kpss.test(X)
```

```
## Warning in kpss.test(X): p-value greater than printed p-value
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  X
## KPSS Level = 0.18423, Truncation lag parameter = 5, p-value = 0.1
```

The Dickey-Fuller statistic (-11.349) is significantly lower than the standard critical values (e.g., -2.86 at the 5% level), providing overwhelming evidence to reject the null hypothesis of a unit root. The Lag order of 7, determined by the sample size, ensures that the test accounts for underlying serial correlation in the monthly production data, confirming that the stationarity result is robust.

The KPSS test confirms the results of the ADF test. The test statistic (KPSS Level = 0.18423) is well below the 5% critical value of 0.463. With a p-value of 0.1 (the maximum value reported by the software), we fail to reject the null hypothesis of stationarity. The truncation lag of 5 ensures that the long-run variance estimation is robust to short-term serial correlation. Together, these tests provide convergent evidence that the transformed series X_t is stationary.

2 Part II: ARMA models

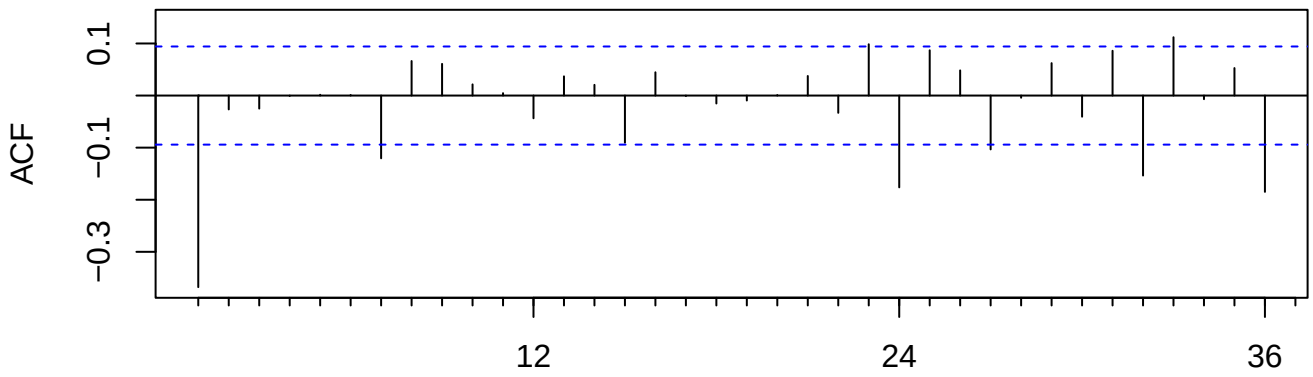
2.1 4. Pick and estimate an ARMA(p,q) model for X_t

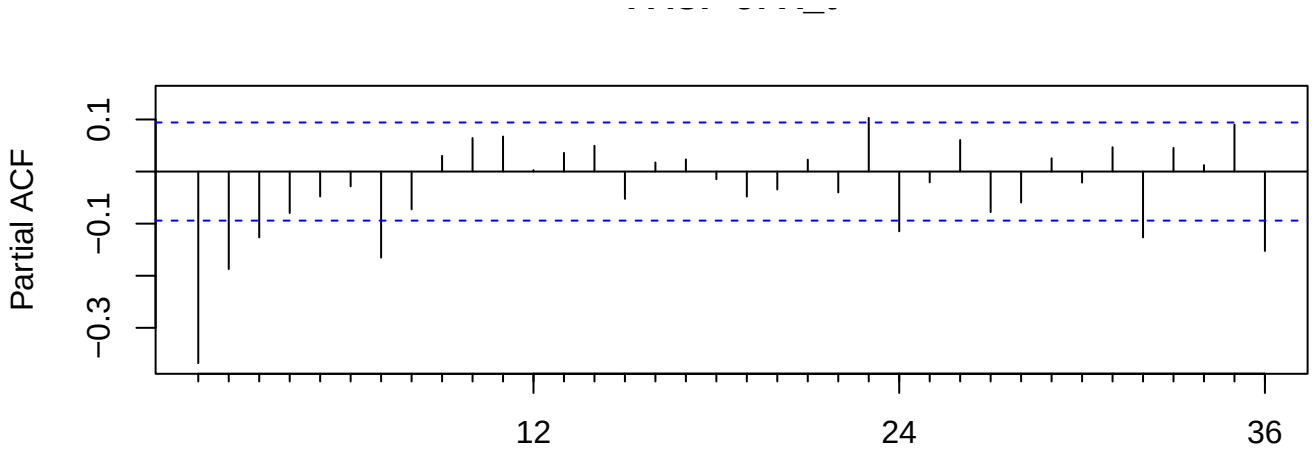
We use ACF/PACF diagnostics and an information-criterion search on a small grid to select a parsimonious ARMA model.

Model identification and selection.

- **Identification logic.** The ACF/PACF of X_t are used as qualitative guidance: short-range dependence suggests an ARMA with relatively small orders rather than a high-order AR (pure PACF cutoff) or pure MA (pure ACF cutoff).
- **Quantitative selection.** We then compare ARMA(p, q) candidates on a grid $(0 \leq p, q \leq 5)$ and select the feasible model minimizing AIC. AIC is preferred here because the goal is forecasting/fit (it penalizes complexity less harshly than BIC).

```r
op <- par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
forecast::Acf(X, lag.max = 36, main = "ACF of X_t")
forecast::Pacf(X, lag.max = 36, main = "PACF of X_t")
```






```
par(op)
```

```
max_p <- 5
max_q <- 5

grid <- expand.grid(p = 0:max_p, q = 0:max_q) %>%
  rowwise() %>%
  mutate(
    fit = list(tryCatch(
      Arima(X, order = c(p, 0, q), include.mean = TRUE),
      error = function(e) NULL
    )),
    aic = ifelse(is.null(fit), NA, AIC(fit)),
    bic = ifelse(is.null(fit), NA, BIC(fit))
  ) %>%
  ungroup() %>%
  arrange(aic)

grid %>%
  filter(!is.na(aic)) %>%
  select(p, q, aic, bic) %>%
  head(5) %>%
```

```
  kable()
```

p	q	aic	bic
4	5	-2477.383	-2432.605
5	5	-2475.192	-2426.344
3	3	-2472.856	-2440.290
4	3	-2471.407	-2434.770
3	4	-2471.403	-2434.766

```
best_row <- grid %>% filter(!is.na(aic)) %>% slice(1)
best_fit <- best_row$fit[[1]]
best_row %>% select(p, q, aic, bic) %>% kable()
```

p	q	aic	bic
4	5	-2477.383	-2432.605

We select the model with the lowest AIC among feasible candidates, which yields ARMA($p = 4, q = 5$) for X_t. The estimated parameters are:

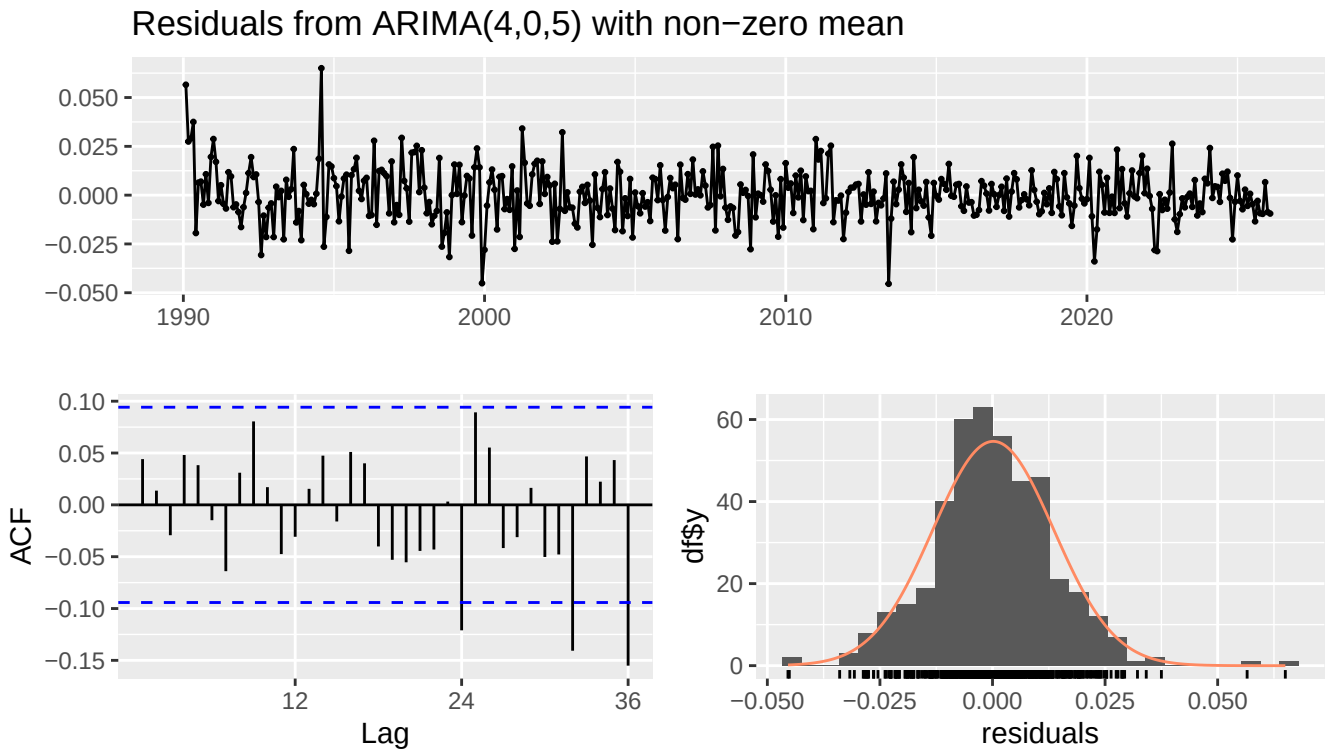
```
coef <- best_fit$coef
se <- sqrt(diag(best_fit$var.coef))
coef_tbl <- tibble::tibble(
  term = names(coef),
  estimate = unname(coef),
  std_error = unname(se),
  t_value = unname(coef / se)
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

coef_tbl %>% kable()
```

term	estimate	std_error	t_value
ar1	0.0540	0.0213	2.5353
ar2	0.6039	0.0237	25.5144
ar3	0.0133	0.0448	0.2964
ar4	-0.9645	0.0174	-55.5530
ma1	-0.6251	0.0733	-8.5317
ma2	-0.6444	0.0658	-9.7895
ma3	0.3518	0.1027	3.4252
ma4	0.9960	0.0484	20.5728
ma5	-0.6083	0.1064	-5.7159
intercept	0.0003	0.0002	1.0657

2.1.1 Diagnostic checks

We validate the fitted model via residual autocorrelation, Ljung-Box test, and a Gaussianity check.

```r
checkresiduals(best_fit, test = FALSE)
```



Residual diagnostics.

- **Whiteness.** The Ljung–Box test checks for remaining autocorrelation in residuals. A non-significant p-value supports the ARMA adequacy (no leftover linear dependence).
- **Gaussianity.** The Jarque–Bera test is a strict check for normality. For many macro series, residuals can be non-Gaussian because of outliers (e.g., crisis months), which affects distributional statements (prediction intervals/regions) more than point forecasts.
- **Here (numbers).** Ljung–Box at lag 24 (df adjusted) gives p-value 0.0528, and Jarque–Bera gives p-value 1.89×10^{-15}.

Overall, the model is considered acceptable for linear dependence if residual autocorrelation is not significant; the Gaussian assumption used later for the confidence region should be interpreted as an approximation if normality is rejected.

2.2 5. Write the ARIMA(p,d,q) model for the chosen series

Let Z_t be the transformed series (log if used) and $X_t = \nabla^d Z_t$ be the stationary series. If the selected ARMA model is ARMA(p, q) for X_t, then the original series follows an ARIMA(p, d, q) model for Z_t. If $Z_t = \log Y_t$, this corresponds to an ARIMA(p, d, q) for the log series, which implies a multiplicative structure on Y_t.

ARIMA representation. With $d = 1$ and the selected ARMA$(p = 4, q = 5)$ for $X_t = \nabla^d Z_t$, the model can be written as

$$\phi(B)\,\nabla^d Z_t = c + \theta(B)\,\varepsilon_t, \qquad \varepsilon_t \sim \text{i.i.d. } (0, \sigma^2),$$

where $\phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ and $\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$. When $Z_t = \log(Y_t)$ and $d = 1$, a non-zero mean in X_t corresponds to a **drift** in $\log(Y_t)$, i.e., a constant average growth component.

3 Part III: Prediction

Denote T the length of the series. We assume the residuals are Gaussian.

3.1 6. Confidence region for (X_{T+1}, X_{T+2})

Let $\hat{\mu} = (\hat{X}_{T+1}, \hat{X}_{T+2})^\top$ be the 2-step-ahead point forecast mean vector, and $\hat{\Sigma}$ the 2×2 forecast error covariance matrix, which quantifies both the uncertainty at each horizon and the correlation between them.

Under the assumption of strictly Gaussian residuals, any linear combination of these residuals remains Gaussian. Consequently, the joint forecast error follows a bivariate normal distribution: $(X_{T+1}, X_{T+2})^T - \hat{\mu} \sim \mathcal{N}_2(\mathbf{0}, \hat{\Sigma})$.

According to the properties of multivariate Gaussian vectors, the standardized quadratic form follows a Chi-squared distribution with 2 degrees of freedom. Therefore, the level-α confidence region takes the shape of an ellipse centered on $\hat{\mu}$:

$$\mathcal{C}_\alpha = \left\{ x \in \mathbb{R}^2 : (x - \hat{\mu})^\top \hat{\Sigma}^{-1} (x - \hat{\mu}) \leq \chi^2_{2,\alpha} \right\}$$

where $\chi^2_{2,\alpha}$ is the quantile of order α of the χ^2 distribution with 2 degrees of freedom.

For a stationary ARMA model, we use its MA(∞) representation with coefficients $\{\psi_j\}_{j \geq 0}$ ($\psi_0 = 1$) to compute the elements of $\hat{\Sigma}$. Since forecast errors are strictly composed of future, unobserved innovations ϵ_t, we obtain:

- $\mathrm{Var}(X_{T+1} - \hat{X}_{T+1}) = \sigma^2$.
- $\mathrm{Var}(X_{T+2} - \hat{X}_{T+2}) = \sigma^2(1 + \psi_1^2)$.
- $\mathrm{Cov}(X_{T+1} - \hat{X}_{T+1}, X_{T+2} - \hat{X}_{T+2}) = \sigma^2 \psi_1$.

where σ^2 is the constant variance of the white noise innovations. This gives us the form of $\hat{\Sigma}$:

$$\hat{\Sigma} = \sigma^2 \begin{pmatrix} 1 & \psi_1 \\ \psi_1 & 1 + \psi_1^2 \end{pmatrix}$$

3.2 7. Hypotheses used to obtain this region

To derive the exact elliptical confidence region $\mathcal{C}_\alpha$, four hypotheses are required:

1. **Correct specification of the ARMA model:** We assume the true Data Generating Process follows the postulated ARMA(p, q) structure. This ensures the theoretical MA(∞) coefficients ψ_j are correct, which is necessary to compute a valid conditional forecast vector $\hat{\mu}$ and covariance matrix $\hat{\Sigma}$.
2. **Strong White Noise (i.i.d. Gaussian innovations):** The unobserved shocks $(\epsilon_t)_{t \in \mathbb{Z}}$ are assumed to be independent and identically distributed according to a Gaussian distribution $\mathcal{N}(0, \sigma^2)$. This ensures the joint forecast error is bivariate normal, allowing the standardized quadratic form to follow a χ^2_2 distribution. Without it, particularly if empirical innovations have heavier tails, the theoretical ellipse will underestimate the true risk.

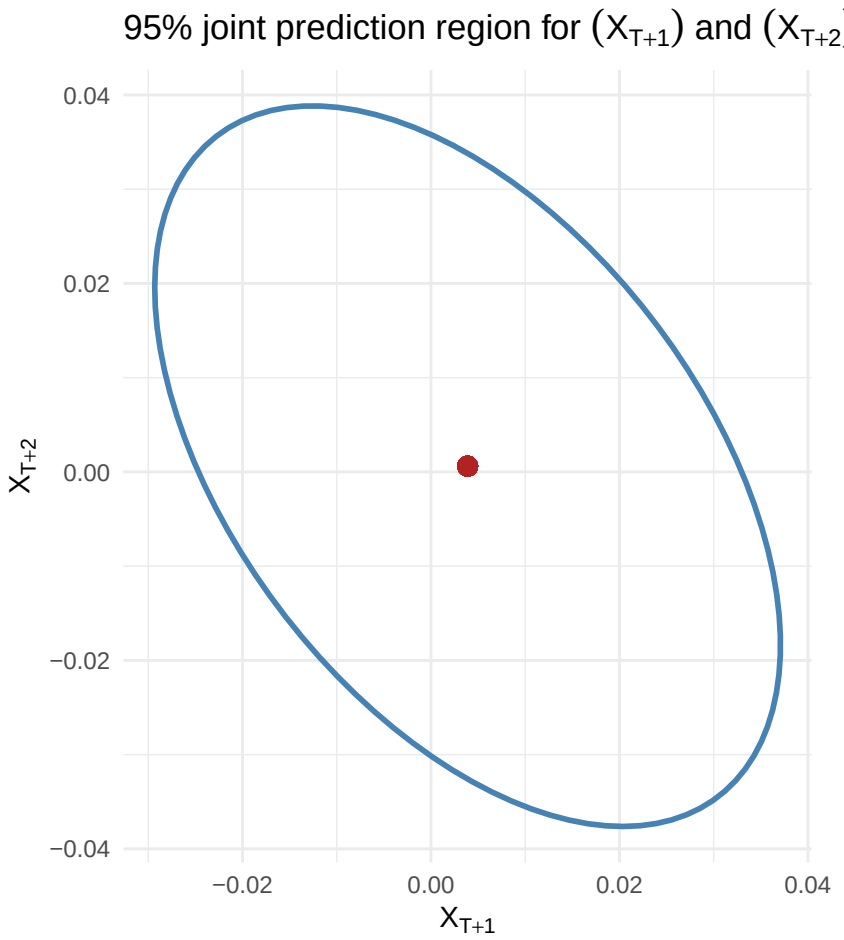
3. **Parameters are treated as known:** The derivation relies on the parameters (ϕ, θ, σ^2) being treated as fixed constants, ignoring the sampling variability of their empirical estimators $(\hat{\phi}, \hat{\theta}, \hat{\sigma}^2)$. This simplification makes the theoretical region slightly too narrow in small samples, but the effect becomes asymptotically negligible as the sample size T grows.

4. **Stationarity and Invertibility:** We require all roots of the autoregressive polynomial $\Phi(z)$ and the moving average polynomial $\Theta(z)$ to lie outside the unit circle. Stationarity guarantees the existence of the Wold representation with a finite variance $(\sum \psi_j^2 < \infty)$, while invertibility allows us to express the unobserved innovations in terms of the observable past sequence $\{X_t, X_{t-1}, \dots\}$, making the forecast computable.

3.3 8. Graphical representation for $\alpha = 95\%$

```
alpha <- 0.95
pred <- predict(best_fit, n.ahead = 2)
mu <- as.numeric(pred$pred)

phi <- if(length(best_fit$model$phi) > 0) best_fit$model$phi else 0
theta <- if(length(best_fit$model$theta) > 0) best_fit$model$theta else 0

psi_coefs <- ARMAtoMA(ar = phi, ma = theta, lag.max = 1)
psi1 <- psi_coefs[1]

sigma2 <- best_fit$sigma2
Sigma <- matrix(
  c(sigma2,         sigma2 * psi1,
    sigma2 * psi1,  sigma2 * (1 + psi1^2)),
  nrow = 2, byrow = TRUE
)

ell_data <- as.data.frame(ellipse(Sigma, centre = mu, level = alpha))
colnames(ell_data) <- c("X_T1", "X_T2")

ggplot(ell_data, aes(x = X_T1, y = X_T2)) +
  geom_path(color = "steelblue", linewidth = 1) +
  geom_point(aes(x = mu[1], y = mu[2]), color = "firebrick", size = 3) +
  labs(
    title = expression(paste("95% joint prediction region for ", (X[T+1]), " and ", (X[T+2]))),
    x = expression(X[T+1]),
    y = expression(X[T+2])
  ) +
  theme_minimal() +
  coord_fixed()
```

```
## Warning in geom_point(aes(x = mu[1], y = mu[2]), color = "firebrick", size = 3): All aesthetics have
## i Please consider using `annotate()` or provide this layer with data containing
##   a single row.
```



The graph displays the 95% joint confidence ellipse for the 2-step-ahead forecasts. Its center corresponds to the conditional expectation vector $\hat{\mu} = (\hat{X}_{T+1}, \hat{X}_{T+2})^\top$, representing the optimal point forecast. The bounding box of the ellipse is taller along the vertical axis than it is wide along the horizontal axis. This visually confirms that the forecast variance strictly increases with the horizon, consistent with the theoretical derivation $\sigma^2(1 + \psi_1^2) > \sigma^2$. Furthermore, the major axis of the ellipse exhibits a negative slope. This orientation reveals a negative covariance between the forecast errors, which algebraically implies that the coefficient ψ_1 is strictly negative. Consequently, if the model overestimates the series at $T + 1$, it is probabilistically more likely to underestimate it at $T + 2$.

3.4 9. Open question: Improving prediction of X_{T+1} using Y_{T+1}

Let $\mathcal{F}_T^X = \sigma(X_1, \ldots, X_T)$ be the filtration generated by the observed history of X. Observing the contemporaneous variable Y_{T+1} improves the forecast of X_{T+1} in terms of Mean Squared Error if and only if Y_{T+1} is not conditionally independent of X_{T+1} given $\mathcal{F}_T^X$. Formally, this requires the conditional expectations to differ:

$$\mathbb{E}[X_{T+1} \mid \mathcal{F}_T^X, Y_{T+1}] \neq \mathbb{E}[X_{T+1} \mid \mathcal{F}_T^X]$$

In a stationary, linear, and Gaussian framework, this condition strictly implies that Y_{T+1} is correlated with the fundamental innovation of X at time $T+1$. Letting $e_{T+1} = X_{T+1} - \mathbb{E}[X_{T+1} \mid \mathcal{F}_T^X]$ be the forecast error, the condition simplifies to $\mathrm{Cov}(e_{T+1}, Y_{T+1}) \neq 0$.

To test this hypothesis practically, one can specify an ARMAX(p, q) model where Y_t acts as an exogenous contemporaneous regressor:

$$\Phi(L)X_t = \beta Y_t + \Theta(L)\varepsilon_t$$

Testing the predictive value of Y_{T+1} amounts to testing the null hypothesis $\mathcal{H}_0 : \beta = 0$. This can be evaluated using a standard asymptotic t-test (or Wald test) on the estimated parameter $\hat{\beta}$, or by conducting a Likelihood Ratio (LR) test comparing the restricted baseline ARMA model against the unrestricted ARMAX model. A rejection of $\mathcal{H}_0$ confirms that including Y_{T+1} significantly reduces the prediction variance of X_{T+1}. # Appendix: Code

```
knitr::opts_chunk$set(
  echo = TRUE,
  warning = TRUE,
  message = TRUE,
  fig.align = "center",
  fig.width = 10,
  fig.height = 6
)


required_pkgs <- c(
  "readr", "dplyr", "ggplot2", "lubridate",
  "forecast", "tseries", "ellipse", "knitr", "ellipse"
)


missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing_pkgs) > 0) {
  install.packages(missing_pkgs, repos = "https://cloud.r-project.org")
}


library(dplyr)
library(ggplot2)
```

```r
library(tseries)

library(forecast)

library(knitr)

library(ellipse)



df <- read.csv("data/valeurs_mensuelles.csv", sep = ";", skip = 4, header = FALSE, stringsAsFactors = F


df <- df[, 1:2]

colnames(df) <- c("date_brute", "valeur")


df <- df[df$valeur != "" & !is.na(df$valeur), ]


# On ajoute "-01" pour créer un format AAAA-MM-JJ lisible par R

df$date <- as.Date(paste0(df$date_brute, "-01"))

df <- df[order(df$date), ]


# On récupère l'année et le mois de début

start_y <- as.numeric(format(df$date[1], "%Y"))

start_m <- as.numeric(format(df$date[1], "%m"))


Y <- ts(df$valeur, start = c(start_y, start_m), frequency = 12)


plot(Y, main = "Série Y : IPI Industries Alimentaires", ylab = "Indice", col = "blue")

Z <- log(Y)

plot(Z, main = "Série Z = log(Y)", ylab = "Indice", col = "green")

X <- diff(Z, differences = 1)
```

```r
plot(X, main = "Série diff(Z)", ylab = "Indice", col = "red")

adf.test(X)

kpss.test(X)

op <- par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))

forecast::Acf(X, lag.max = 36, main = "ACF of X_t")

forecast::Pacf(X, lag.max = 36, main = "PACF of X_t")

par(op)

max_p <- 5

max_q <- 5


grid <- expand.grid(p = 0:max_p, q = 0:max_q) %>%

  rowwise() %>%

  mutate(

    fit = list(tryCatch(

      Arima(X, order = c(p, 0, q), include.mean = TRUE),

      error = function(e) NULL

    )),

    aic = ifelse(is.null(fit), NA, AIC(fit)),

    bic = ifelse(is.null(fit), NA, BIC(fit))

  ) %>%

  ungroup() %>%

  arrange(aic)


grid %>%

  filter(!is.na(aic)) %>%

  select(p, q, aic, bic) %>%

  head(5) %>%
```

```
  kable()


best_row <- grid %>% filter(!is.na(aic)) %>% slice(1)

best_fit <- best_row$fit[[1]]

best_row %>% select(p, q, aic, bic) %>% kable()

coef <- best_fit$coef

se <- sqrt(diag(best_fit$var.coef))

coef_tbl <- tibble::tibble(

  term = names(coef),

  estimate = unname(coef),

  std_error = unname(se),

  t_value = unname(coef / se)

) %>%

  mutate(across(where(is.numeric), ~ round(.x, 4)))


coef_tbl %>% kable()

checkresiduals(best_fit, test = FALSE)

res <- residuals(best_fit)

lb_test <- Box.test(res, lag = 24, type = "Ljung-Box", fitdf = sum(best_fit$arma[1:4]))

jb_test <- tseries::jarque.bera.test(res)

alpha <- 0.95

pred <- predict(best_fit, n.ahead = 2)

mu <- as.numeric(pred$pred)


phi <- if(length(best_fit$model$phi) > 0) best_fit$model$phi else 0

theta <- if(length(best_fit$model$theta) > 0) best_fit$model$theta else 0
```

```r
psi_coefs <- ARMAtoMA(ar = phi, ma = theta, lag.max = 1)

psi1 <- psi_coefs[1]


sigma2 <- best_fit$sigma2

Sigma <- matrix(

  c(sigma2,          sigma2 * psi1,

    sigma2 * psi1,   sigma2 * (1 + psi1^2)),

  nrow = 2, byrow = TRUE

)


ell_data <- as.data.frame(ellipse(Sigma, centre = mu, level = alpha))

colnames(ell_data) <- c("X_T1", "X_T2")


ggplot(ell_data, aes(x = X_T1, y = X_T2)) +

  geom_path(color = "steelblue", linewidth = 1) +

  geom_point(aes(x = mu[1], y = mu[2]), color = "firebrick", size = 3) +

  labs(

    title = expression(paste("95% joint prediction region for ", (X[T+1]), " and ", (X[T+2]))),

    x = expression(X[T+1]),

    y = expression(X[T+2])

  ) +

  theme_minimal() +

  coord_fixed()

code_chunks <- knitr::knit_code$get()

code_text <- unlist(code_chunks, use.names = FALSE)


cat("```r\n")
```

```
cat(paste(code_text, collapse = "\n\n"))

cat("\n```\n")
```